計算複雜度與近似

當問題規模增加，如何判斷求解是否有效率？我們先以輸入編碼長度衡量時間，再研究判定問題、驗證與多項式時間歸約。

接著研究最佳化問題的近似演算法，以可行解、上下界及反例理解保證的適用條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

判定問題 → P 與 NP → 歸約方向 → NP 完全性 → 近似比與具體演算法。

區分證明困難度的歸約與求近似解的方法；NP 完全性本身不證明 P ≠ NP，也不給指數時間下界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先把問題寫成語言

本節從輸入位元數出發，建立 P、NP 與 polynomial-time many-one reduction。目標是能獨立證明 3SAT $\leq_p$ CLIQUE，而不是背歸約箭頭。

前置知識為布林公式、無向簡單圖、漸近界與全稱／存在量詞。

學完應能回答「證書驗得快」為何不表示「證書找得快」，並指出每項 NP-completeness 證明的兩個義務。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

決策、搜尋與最佳化先分清

形式	CLIQUE 例子	輸出
決策 decision	是否存在至少 k 個頂點的 clique？	yes／no
搜尋 search	找出 k-clique，如存在	頂點集合或不存在
最佳化 optimization	求最大 clique 大小 ω(G)	整數

本節 P、NP 以決策語言定義。最佳化的多項式演算法能解決決策版，只需比較 $\omega(G) \geq k$。反向還需額外構造，不能只說兩者是同一個問題。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

輸入長度不是數值大小

設 x 為完整編碼，N = |x| 為位元數。整數 t ≥ 1 的二進位長度為 $\lfloor \log_2 t \rfloor + 1$。圖的編碼也要包含頂點與邊，門檻 k 也在輸入內。

若 t = 2^m，二進位只需 m+1 位。執行 t 次迴圈需 Θ(2^m) 步，不能因為寫成 O(t) 就稱為多項式時間。若改用 unary，輸入有 t 個符號；那已是不同編碼。

圖的 adjacency matrix 與顯式 adjacency list 可互相以多項式時間轉換。允許這類轉換不代表允許把二進位數值展成 unary。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

輸入長度不是數值大小 （續）

同一數值，不同輸入長度

t = 2^m

二進位 binary

1000...000

m + 1 位元

t 次迴圈 = Θ(2^m)

一元 unary

1111...111

2^m 個符號

t 次迴圈 = Θ(輸入長度)

多項式必須相對於完整編碼長度 N = |x|。

O(t) 不代表 O(poly(log t))。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從編碼集合定義 P

決策語言 L ⊆ {0,1}* 收集所有 yes instance 的編碼。約定錯誤編碼不在 L，演算法必須在多項式時間拒絕。

$$L \in P \iff \exists M\ \exists p \text{ polynomial } \forall x:\ M(x) \text{ halts within } p(|x|) \text{ and } [M(x) = 1 \iff x \in L].$$

M 是 deterministic decider。時間界須對每個輸入成立，包含 no instance。PATH = {⟨G,s,t,k⟩ | 存在至多 k 條邊的 s 到 t 路徑} 可用 BFS 求最短距離後比較，故在 P。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP 的定義包含證書長度

存在 deterministic verifier V 與多項式 p、q，使

$$x \in L \iff \exists c,\ |c| \le p(|x|) : V(x, c) = 1.$$

V 在 O(q(|x|+|c|)) 時間內停止。因為證書長度也受 p 限制，合法候選的驗證時間才是 |x| 的多項式。可先拒絕超長證書。

NP 是 nondeterministic polynomial time，不是 non-polynomial。定義保證 yes instance 有短證書，沒有保證能在多項式時間找到它。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完備性與健全性是兩個量詞

義務	正確敘述	不能換成
完備性 completeness	$x \in L \Rightarrow$ 存在短 c 使 V 接受	每份證書都接受
健全性 soundness	$x \notin L \Rightarrow$ 所有允許的 c 都使 V 拒絕	存在一份被拒絕的 c

每份合法輸入與允許長度的證書都必須讓 V 停止。許多 validator 只檢查資料格式；NP verifier 還要證明證書確實支持該 yes 答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完備性與健全性是兩個量詞 （續）

Verifier 的完備性與健全性

完備性 completeness

$x \in L$ → 至少一份短證書 c → $V(x,c) = 1$

存在證書，不表示每份證書都成功。

健全性 soundness

$x \notin L$ → 所有允許的證書 c → $V(x,c) = 0$

不能只證明「有一份證書被拒絕」。

$|c| \le p(|x|)$，驗證時間 $\le q(|x| + |c|)$，p、q 為多項式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CLIQUE 的 verifier 寫到可執行

輸入為簡單無向圖 G 與二進位非負整數 k。若 $k > n$，直接拒絕。$k = 0$ 時接受空證書。其餘證書是恰好 k 個不同頂點編號。

1. 檢查編碼、證書長度、編號範圍與互異性。
2. 對所有 $i < j$，檢查 $\{c_i, c_j\} \in E$。
3. 全部通過才接受。

證書長度 $O(k \log(n+1)) \leq O(n \log(n+1))$。建好 adjacency matrix 後需 $O(n^2+k^2)$ 次基本圖操作，位元操作成本仍為輸入長度的多項式。若有 clique，可取其 k 個頂點；若 verifier 接受，所有頂點對均相鄰，確實是 clique。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

已知 P ⊆ NP；等號仍未知

若 L ∈ P，令 V(x,c) 直接執行 L 的 decider 並忽略 c。每個 yes instance 皆可用空證書，no instance 不會被任何證書誤接受。因此 P ⊆ NP。

目前未解的是 NP ⊆ P 是否成立，亦即 P = NP 是否成立。不能畫出 P 與 NP-complete 分離的圖而不註明假設 P ≠ NP。

NP-complete 的已知歸約定理不等於已證 P ≠ NP；指數時間暴力演算法也不能證明所有演算法都需要指數時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

many-one 歸約保留 yes 與 no

A ≤p B 指存在總函數 f，可在輸入長度的多項式時間計算，且對所有 x，

$$x \in A \iff f(x) \in B.$$

f 必須在多項式時間計算，不能假設可免費求解 A。若 A∈P，且 B 非空且非全集，可先判定 A，再依結果輸出 B 的固定 yes 或 no 字串。這是輸出單一字串的 many-one reduction，不是多次 oracle 查詢。

所有字串均須滿足等價式。若來源拒絕錯誤編碼且目標有固定 no 字串，可將錯誤編碼映至它；一般目標未必有 no 字串，例如 B={0,1}*。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

歸約方向由求解器接線決定

若 A ≤p B，則「會解 B」可推出「會解 A」。演算法為 x → f(x) → B-decider → A 的答案。

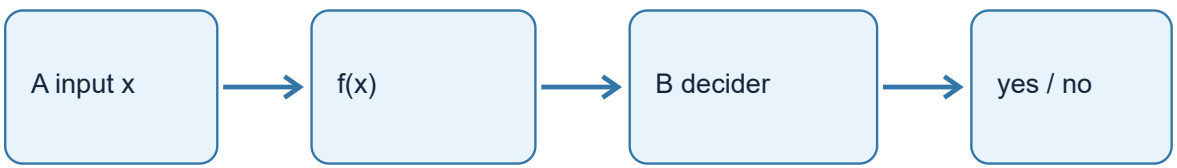
因此 B ∈ P ⇒ A ∈ P。取逆否可得 A ∉ P ⇒ B ∉ P。不能由 A ∈ P 推得 B ∈ P，也不能由 B 很難推得 A 很難。

要證明目標 B 很難，應由已知難的 A 歸約到 B。若只證 B ≤p A，僅表示 B 可借用 A 的求解器。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

歸約方向由求解器接線決定 （續）

A ≤p B 讓 A 使用 B 的求解器



$x \in A \Leftrightarrow f(x) \in B$

$B \in P \Rightarrow A \in P$

證明 B 很難，從已知難的 A 指向 B。

f 必須在多項式時間直接建構 instance。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何時間與輸出長度都要追蹤

設 f 的時間為 O(N^a)，且輸出長度 |f(x)| ≤ C N^a。若 B-decider 在長度 M 的輸入上需 O(M^b)，則組合時間為

$$O(N^a) + O((CN^a)^b) = O(N^a + N^{ab}).$$

a、b、C 為固定常數，故仍為多項式。關鍵是標準逐位元輸出模型下，輸出不可能比執行時間還長。

若允許把指數大小的 B instance 以「壓縮描述」當作同一輸入，B 的原時間界不能直接套用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

many-one 歸約的傳遞性

若 A ≤p B 經 f，且 B ≤p C 經 g，定義 h = g ∘ f。

語義上，x ∈ A ⇔ f(x) ∈ B ⇔ g(f(x)) ∈ C。兩個雙條件缺一不可。

若 f 的時間 O(N^a)，g 的時間 O(M^b)，則 |f(x)| = O(N^a)，h 的時間 O(N^a + N^(ab))。因此 A ≤p C。

這證明容許串接固定數量的多項式歸約。若重複 N 次且每次長度平方，最終長度可能 N^(2^N)，不能直接套「多項式次數的呼叫仍為多項式」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP-hard 與 NP-complete 的條件

名稱	條件
B 為 NP-hard	對每個 $A \in NP$，$A \leq_p B$
B 為 NP-complete	B 為 NP-hard 且 $B \in NP$

若已知 A 是 NP-hard，證 $A \leq_p B$ 即可由傳遞性證 B NP-hard。再獨立給 B 的短證書與多項式 verifier，才能得 NP-complete。

NP-hard 本身不保證 $B \in NP$，亦不保證 B 可判定。本節以決策語言定義；談最佳化問題 NP-hard 時，須先說明採何種決策或求解歸約。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3SAT 到 CLIQUE 的建構

令 $F = C_1 \wedge \cdots \wedge C_m$，每個子句含三個 literal occurrences，本節允許重複。CLRS 的三個相異 literals 版本是其子集合，故 NP-hardness 仍適用，assignment verifier 也仍為多項式。3SAT 的 NP-completeness 採用 CLRS34 已知定理，此處證明目標 CLIQUE。

- 每個 occurrence 建一個頂點 v_{ij}，記錄其子句 i 與 literal ℓ_{ij}。
- 同一子句的頂點完全不連邊。
- 不同子句的兩頂點，當且僅當 literals 不是互補的 x 與 ¬x 才連邊。
- 輸出 $\langle G, k=m \rangle$。

重複出現的相同 literal 仍是不同頂點。跨子句的相同 literal 彼此相容，必須連邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把實際公式畫成完整圖

取 F = (x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ z) ∧ (x ∨ y ∨ ¬z)。

頂點編號依子句順序為 1:x、2:¬y、3:z；4:¬x、5:y、6:z；7:x、8:y、9:¬z。完整邊集如下，圖中每條灰線或綠線都是真實邊。

E = {15,16,24,26,34,35,36,17,18,19,27,29,37,38,48,49,57,58,59,67,68}，共 21 條。x=y=z=true 時，可選 {1,5,7}，三條邊 15、17、57 都存在。

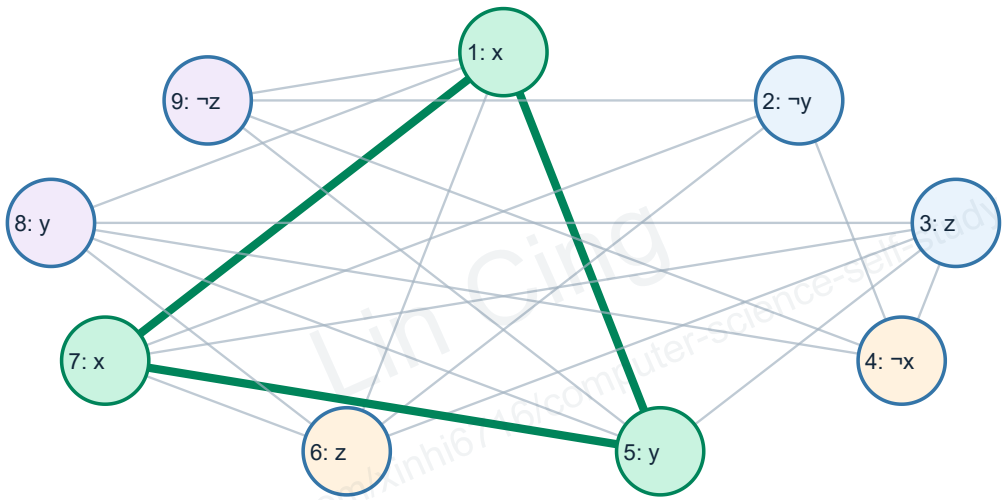
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把實際公式畫成完整圖 （續）

3SAT → CLIQUE，完整 9 點圖

F = (x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ z) ∧ (x ∨ y ∨ ¬z)

C1 = {1,2,3} C2 = {4,5,6} C3 = {7,8,9}



綠色 {1,5,7} 是 3-clique；群內無邊；互補 literals 無邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正向證明由 assignment 選 clique

假設 F 可滿足，固定一個滿足 assignment。每個子句至少有一個真 literal，從每個子句各選一個對應頂點，得到 m 個頂點 S。

任意兩個選中頂點來自不同子句，且其 literals 同時為真，不可能是互補的 x 與 ¬x。依建構規則兩者相鄰。

所以 S 中每一對頂點都相鄰，$|S|=m$，為 m-clique。這證明 $F \in 3SAT \Rightarrow \langle G,m\rangle \in CLIQUE$。歸約建圖本身沒有使用這個 assignment，它只在正確性證明中出現。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

反向證明由 clique 還原 assignment

假設 G 有大小 m 的 clique S。每個子句群內沒有邊，所以 S 在每群最多選一點。共 m 群而有 m 點，故每群恰選一點。

S 的任兩點相鄰，所以所選 literals 不含互補對。令選中的正 literal 之變數為 true，負 literal 之變數為 false。沒有變數被要求同時為 true 與 false。未選到的變數任意賦值。

每個子句都含一個選中的真 literal，故 F 可滿足。這證明 $\langle G,m\rangle \in \text{CLIQUE} \Rightarrow F \in \text{3SAT}$，完成雙條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

建圖成本與邊界必須相容

輸入長度 N 至少與 occurrence 數同階。m 個三文字子句產生 3m 個頂點，至多 9・m(m−1)/2 條邊。比較所有跨群 pairs，連同變數編號的位元比較，加上輸出邊編號的成本，$O(N^2 \log(N+1))$ 位元操作足夠；顯式邊串列大小 $O(m^2 \log(m+1))$，也是 N 的多項式。

邊界	約定與理由
m=0	空 conjunction 為 true；G 空、k=0，空 clique 存在
同變數正負跨群	不連邊，避免矛盾 assignment
單子句含 x 與 ¬x	群內仍不連邊；只須挑一個真 literal
重複 literal	保留 occurrence 頂點，不合併
不合法 3CNF	映到固定 no instance ⟨空圖,1⟩

若 CLIQUE 規格只允許 k≥1，m=0 應另映到固定 yes instance $\langle K_1, 1\rangle$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

現在才可宣告 CLIQUE NP-complete

1. 已知 3SAT 是 NP-complete。
2. 已構造多項式時間總函數 f，並證 F 可滿足 $\Leftrightarrow$ f(F) 有 m-clique。
3. 所以 3SAT $\leq_p$ CLIQUE。由歸約傳遞性，CLIQUE 為 NP-hard。
4. 前述頂點證書與 verifier 證 CLIQUE $\in$ NP。
5. 合起來得 CLIQUE 為 NP-complete。

若有人提出 CLIQUE 的多項式時間演算法，便由歸約得所有 NP 語言皆在 P，配合 P $\subseteq$ NP 得 P=NP。這是條件定理，沒有證明 P$\neq$NP。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

決策 oracle 如何找最佳值與證書

對 CLIQUE，固定圖 G，謂詞「有至少 k 個頂點的 clique」隨 k 單調。於 [0,n] 二分搜尋，用 O(log(n+1)) 次 decision 呼叫可得 ω(G)。這是多次呼叫的求解方法，不是剛才的一次 many-one reduction。

已知 G 有 k-clique。依序試刪頂點 v，若 G−v 仍有 k-clique 就刪除，否則保留。至多 n 次呼叫後圖為 inclusion-minimal 的 yes instance，恰有 k 個頂點，得到證書。

理由是任取一個 k-clique，若尚有 clique 外的頂點，它仍可刪除。某頂點一旦不可刪，往後刪除其他頂點也不會讓它變得可刪。此處每次子圖長度不增，故若 decision 在 P，整個搜尋亦為多項式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從驗證器到第一個 NP 完全問題

給定布林電路 C，CIRCUIT-SAT 詢問是否存在輸入 y 令 $C(y) = 1$。它屬於 NP：把輸入指派作為 witness，依拓撲順序計算每個 gate，時間與編碼長度成多項式。

要證明 NP-hard，不能只從某個尚未知為 NP-hard 的問題歸約。固定任意語言 $L \in \mathrm{NP}$，取其確定性多項式時間 verifier V、多項式界 p, T，使 $x \in L \iff \exists y \in \{0,1\}^{p(|x|)} : V(x, y) = 1$。長度不足的 witness 可用長度欄位與 padding 編碼。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把一次執行攤開成電路

固定 x，令 $n = |x|$，使用固定且對所有輸入一致的編碼方式。把 verifier 的記憶體、讀寫頭、控制狀態與輸入寫成長度 $s(n) = \mathrm{poly}(n)$ 的 configuration；最多 $T(n) = \mathrm{poly}(n)$ 步。接收後以吸收狀態維持接收，恰好執行 $T(n)$ 步也不改變答案。

選用單帶圖靈機模型時，每個 cell 的下一步狀態由其自身及固定範圍鄰居決定；有限的局部轉移表可由常數大小的布林 gate 電路實作。對每一步複製一層、對每個 cell 複製局部電路，得到 $O(T(n)s(n))$ 個 gate。其他合理模型也須先給出多項式模擬，不能把任意 RAM 操作視為免費常數大小電路。

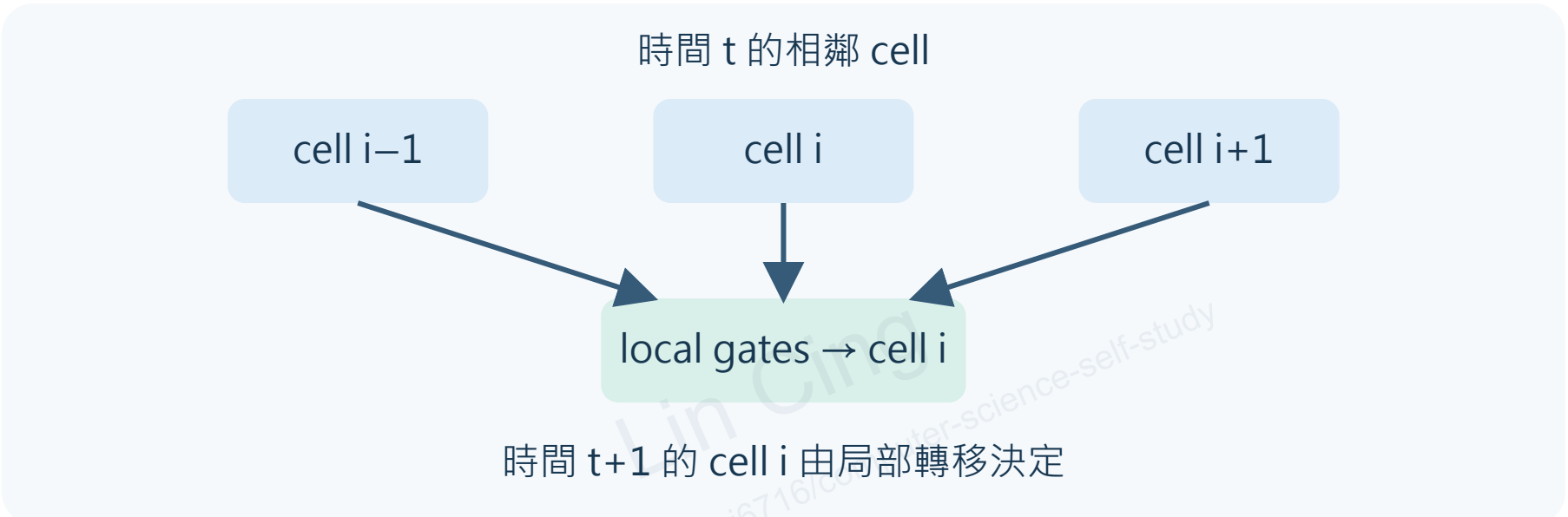
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把一次執行攤開成電路 （續）



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

局部更新如何接成下一層



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正確性：逐層維持同一個 configuration

初始層將 x 的每一位固定為常數，y 的 $p(n)$ 位保留為自由輸入，其餘位置寫入初始空白與控制狀態。若採不同長度的 witness，固定長度編碼內含合法長度檢查；非法編碼輸出 0。

對任意自由輸入 y，第 0 層就是 $V(x, y)$ 的初始 configuration。假設第 t 層等於實際第 t 步，局部 gate 依同一轉移規則給出第 $t+1$ 步，故歸納成立。最後 output 取接收狀態位，因此 $C_x(y) = 1 \iff V(x, y) = 1$；存在滿足輸入恰好等價於 $x \in L$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼這是一個多項式時間歸約

L 一旦固定，verifier、局部規則、常數及多項式上界也固定；它們可寫進建構器。對每個 x，建構器算出 $T(n), s(n)$、逐層輸出 gate 描述，並把 x 硬接為常數；時間和輸出長度皆為 $\mathrm{poly}(n)$。這是 uniform construction：不是僅宣稱「每個長度存在一個小電路」。

因此任意 $L \in NP$ 都有 $L \leq_p \mathrm{CIRCUIT\text{-}SAT}$；連同 membership，CIRCUIT-SAT 是 NP-complete。電路可含共享 wire；不必展開為可能指數大的公式樹。歸約輸出的是電路本身，而不是先求出 witness。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從電路到布林可滿足性

CIRCUIT-SAT 到 SAT 的局部 gate 約束

為每條 wire（可共享同一 gate output）設一變數，並把輸出 wire z_{out} 本身作為 unit clause，要求它為真。每個 gate 加入輸出等於 gate 函數的條件：

Gate 關係	等價 CNF clauses
z = u AND v	(¬z∨u) AND (¬z∨v) AND (z∨¬u∨¬v)
z = u OR v	(z∨¬u) AND (z∨¬v) AND (¬z∨u∨v)
z = NOT u	(z∨u) AND (¬z∨¬u)

例如 AND 的前兩 clauses 強迫 z=true 時 u,v 皆真；最後一 clause 強迫 u=v=true 時 z=true。OR 與 NOT 可同樣分真假兩向直接檢查。常數 wires 用 unit clause 固定。

電路接受輸入可延伸為所有 wire 的真值，滿足每個局部 clause 與 output clause。反向滿足公式時，局部等價式沿拓撲序迫使每個 wire 等於實際計算值，output=true，所以原電路可滿足。每 gate 至多三 clauses，總長 O(gates+wires)，不把共享子電路反覆展開成指數長樹狀公式。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

SAT 到 3-CNF 的 parse-tree gadget

任意公式先解析成 NOT、一輸入或 AND/OR 二輸入樹；例如 implication 可用 $\neg a \vee b$ 改寫，其他固定 arity connective 亦用固定大小 truth-table gadget。為每個內節點設新變數，使用前頁 gate clauses，根變數設真。若同一節點輸入重複，先在所得 clauses 去除重複 literal；含 l 與 $\neg l$ 的 clause 是 tautology，可整個刪掉。

此時 clauses 長度至多三。若要求每 clause 恰含三個相異變數，逐 clause 使用新的 padding variables：

- 兩 literal $(a \vee b)$ 改為 $(a \vee b \vee p) \text{ AND } (a \vee b \vee \neg p)$。$a \vee b$ 真則任意 p 都可；若假，兩式分別強迫 p 與 $\neg p$，矛盾。
- 一 literal a 改為四式 $(a \vee p \vee q)$、$(a \vee p \vee \neg q)$、$(a \vee \neg p \vee q)$、$(a \vee \neg p \vee \neg q)$ 的 AND。a 真則皆真；a 假時，任何 p,q 指派都令其中一 clause 為假。
- 空 clause 表示不滿足，直接輸出上述 padding 過的 $z \text{ AND } \neg z$；空 AND 表示恆真，輸出固定可滿足的三變數 clause。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

SAT 到 3-CNF 的 parse-tree gadget （續）

原 assignment 可用每個 parse node 的真值延伸；反向沿樹恢復所有節點語義。padding 保留每 clause 的真假要求，因此整體只保留 equisatisfiability，而非把加入新變數後的公式當作原變數的逐字同一函數。每節點常數 clauses，每 clause 至多擴成四個，所以大小與建構時間皆多項式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

長 clause 拆分與可追蹤 gate 例

若已得到長 clause $l_1 \vee \cdots \vee l_r$，$r > 3$，也可用 chain：

$$(l_1 \vee l_2 \vee y_1) \wedge (\neg y_1 \vee l_3 \vee y_2) \\ \wedge \cdots \wedge (\neg y_{r-3} \vee l_{r-1} \vee l_r).$$

若所有 l_i 都假，首式逼 $y_1 = \text{true}$，中間逐次逼下一個 $y_j = \text{true}$，末式卻要求最後 $y_{r-3} = \text{false}$，矛盾。若至少一個 l_i 真，令 y_j 表示前 $j+1$ 個 literals 是否全假，即可逐式滿足。此拆分用 $r-3$ 新變數、$r-2$ clauses；不能直接用 distributivity 把任意公式轉 CNF，那可能產生指數長度。

例 F=(a∨b) AND ¬a，設 u=a∨b、v=¬a、z=u AND v，給三組 gate clauses 並加 z。a=false,b=true 時 u=v=z=true，滿足所有約束。若 a=true，NOT gadget 強制 v=false，而 z 與 AND gadget要求 v=true，故這類指派不能假造接受。這同時示範 witness 延伸與反解。

原書 3-CNF 定義為三個相異 literals；本教材的主要 reduction 進一步可規格化為三個相異變數。出現重複 literal 的輸入仍可先依上述去重、padding 規則處理，不影響後續 CLIQUE occurrence 構造。

CLIQUE、INDEPENDENT-SET 與 VERTEX-COVER

independent set S 是圖中任兩點都沒有 edge 的集合。vertex cover C 是每條 edge 至少一端在 C 的集合。

對 n 點、無 self-loop 的無向圖 G=(V,E)，令 $\overline{G} = (V, \{uv : u \neq v,\ uv \notin E\})$。同一 pair 在 G 有 edge 若且唯若在 $\overline{G}$ 沒有 edge。因此：

$$C \text{ is a clique in } G \iff C \text{ is an independent set in } \overline{G}.$$

若 S 是 independent set，則 $V \setminus S$ 是 vertex cover。假設有 $\overline{G}$ 的 edge 未被 $V \setminus S$ 覆蓋，它的兩端都在 S，違反 S independent。反向若 $V \setminus S$ 是 vertex cover，任何 $\overline{G}$ edge 不可能兩端都在 S；所以 S independent，也是在 G 的 clique。

令 $k \in \{0, \ldots, n\}$。把上面兩步串起，得到

$\text{CLIQUE}(G, k) \iff \text{VERTEX-COVER}(\overline{G}, n-k)$。

$(G, k) \mapsto (\overline{G}, n-k)$ 與反方向的 $(H, \ell) \mapsto (\overline{H}, n-\ell)$ 都以 $O(n^2)$ 次配對檢查建立補圖。這裡CLIQUE可說「至少k點」或「恰k點」，因較大clique任取k點仍是clique；VC採「至多n−k點」，較小cover可補點至恰n−k點，所以同一iff成立。這需0≤k≤n；範圍外門檻先直接判定並映射至固定yes/no實例，不能產生負大小certificate。

Lin Cing
http://github.com/xinhi6716/computer-science-self-study

CLIQUE 與 VERTEX-COVER 的可追蹤例

令 G 有頂點 {1,2,3}，edges {12,23}，取 k=2。{1,2} 在 G 是 clique。

補圖 $\overline{G}$ 只有 edge {13}。$\{1,2\}$ 在 $\overline{G}$ 是 independent set，因為 1 與 2 沒有 edge。其補集 $\{3\}$ 是 $\overline{G}$ 的 vertex cover，大小為 $3-2=1$，且它覆蓋唯一 edge 13。

轉換是 $\mathrm{VC}(\overline{G}, n-k)$。所以 $\mathrm{CLIQUE}(G,k) \iff \mathrm{VERTEX\text{-}COVER}(\overline{G}, n-k)$。這個等價直接給出雙向約化：complement 本身就是 polynomial-time、可逆的 encoding transformation。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從頂點覆蓋到 Hamiltonian cycle

VERTEX-COVER 到 HAMILTONIAN-CYCLE 的端口 gadget

HC certificate 是全部頂點的排列，逐邊與閉合檢查給 NP membership。下面完整建立 NP-hardness，不需要平面 crossing gadget；圖中交叉畫線不代表交叉處是頂點。

對原圖每邊 {u,v} 建十二個新點 a1,...,a6、b1,...,b6，其中 ai=[u,v,i]、bi=[v,u,i]。內邊共十四條：

- 兩條鏈 a1-a2-a3-a4-a5-a6 與 b1-b2-b3-b4-b5-b6，共十條。
- 四條交叉連結 a1-b3、a3-b1、a4-b6、a6-b4。

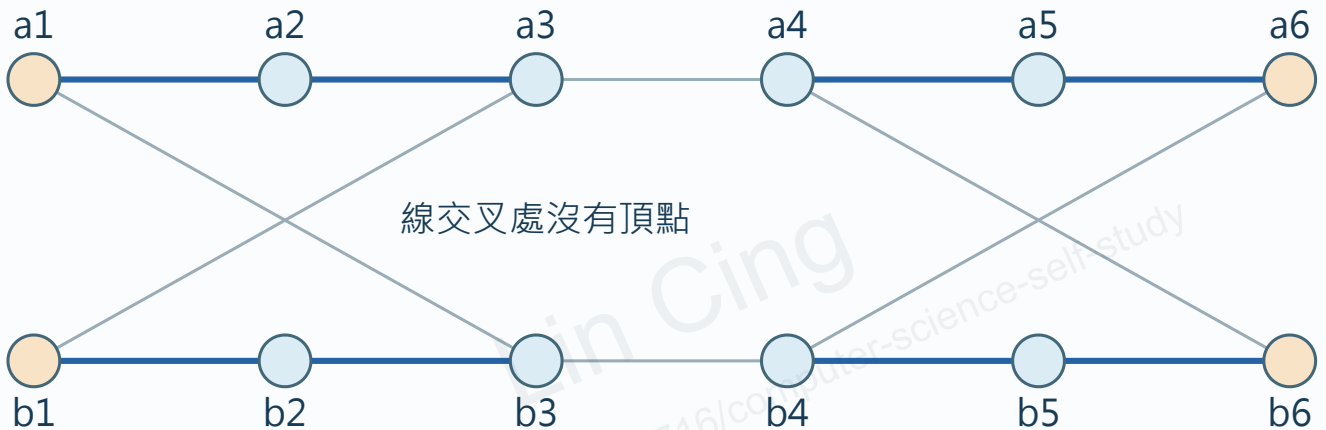
只有四個端口 a1,a6,b1,b6 可接 gadget 外部。其他八點在最後 HC 中必有兩條內邊。a2,b2,a5,b5 在 gadget 本身 degree=2，故它們的八條相鄰鏈邊被迫選入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從頂點覆蓋到 Hamiltonian cycle （續）

HC gadget：只有 a1、a6、b1、b6 能接外部

$a_i=[u,v,i]$；$b_i=[v,u,i]$



藍色8邊由 a2、a5、b2、b5 的 degree=2 強迫選入；灰色6邊仍待決定。

兩條中邊 a3–a4、b3–b4 決定其餘4條cross edges；橘色為4個端口。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

gadget 只有三種合法的覆蓋方式

餘下選擇由中間兩邊 a3-a4、b3-b4 決定。每個內部點仍需 degree=2，故這兩邊一旦選或不選，四條交叉連結的使用亦被迫確定。

中邊選擇	對全部十二點的路徑覆蓋
兩條都選	a1-a2-a3-a4-a5-a6，以及 b1-b2-b3-b4-b5-b6，兩條 disjoint paths
只選 b3-b4	a1-a2-a3-b1-b2-b3-b4-b5-b6-a4-a5-a6，一條 a1 到 a6 的完整 path
只選 a3-a4	對稱的一條 b1 到 b6 的完整 path
兩條都不選	上下各形成一個封閉六點 cycle，與外部無法連成單一 HC，因此禁止

這四種窮盡所有可行內度配置，故合法 HC 只能使用前三種。尤其不能從 a 端口進來卻沿 b 的外部鏈離開；也不能讓十二點完全無人覆蓋。此局部事實正好表示原邊至少有一個端點被選。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

中邊如何強迫端口的外部使用

令x表示是否選a3−a4，y表示是否選b3−b4，值為0或1。內點a3已有a2−a3，所以a3−b1必取1−x；a4同理使a4−b6取1−x。b3,b4則使a1−b3與a6−b4取1−y。

因此a1,a6的內部度都是2−y，還需要的外部度為y；b1,b6的外部度為x。端口不可能一邊連外、一邊封閉：同一原頂點側的兩端總是同時接外或同時不接外。

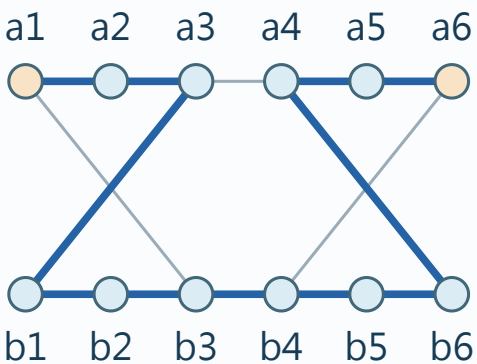
x=y=0時，下列兩個六點cycle已各自用滿度2：a1−a2−a3−b1−b2−b3−a1，以及a4−a5−a6−b4−b5−b6−a4。它們不能接到selector，故不可能是整張圖的單一HC。其餘三種配置的全部路徑如下圖，逐點可核對沒有漏點或換側離開。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

中邊如何強迫端口的外部使用 (續)

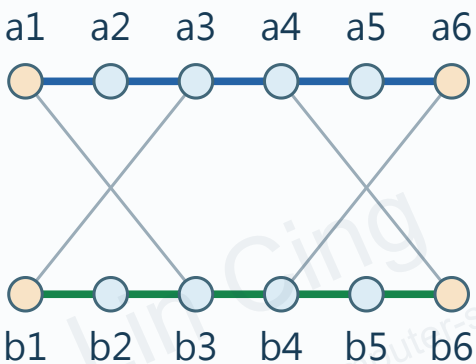
三種合法模式都不會跨原頂點身分離開

只走u側：12點



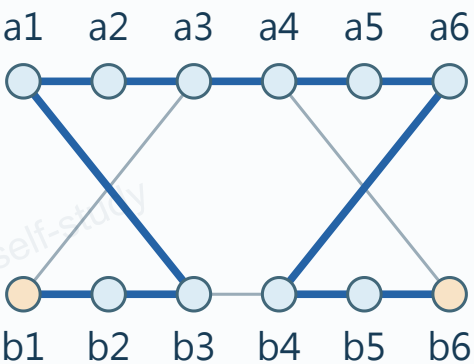
x=0，y=1

u、v各走6點



x=1，y=1

只走v側：12點



x=1，y=0

灰邊未選；藍 / 綠是各條路徑；橘色端口各需一條外邊補到 degree=2。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

串接原頂點的 incident gadgets 與 selectors

VERTEX-COVER 輸入採「大小至多 k」。先移除孤點，不影響最小 cover。k<0 或 k=0 且有邊，輸出固定 no HC instance；無邊或 k 大於等於剩餘頂點數，輸出固定 yes triangle。以下可假設無孤點且 $1 \le k < |V|$；一個較小 cover 可以補入其他非孤點成恰 k 點。

對每個原頂點 u，任排其鄰居 u(1),...,u(d)。把各 gadget 的 u 側依序串接：加入 [u,u(i),6]-[u,u(i+1),1]，i=1,...,d-1。這些連結使一條外部路徑一旦選 u，便沿它全部 incident gadgets 走到末端。

另建 k 個 selector s1,...,sk。每個 selector 都接到每個 u 鏈的首端 [u,u(1),1] 與末端 [u,u(d),6]，沒有其他外邊。圖共有 12|E|+k 點；十四內邊乘 |E|，串鏈至多 2|E|，selector 連邊 2k|V|，全為多項式規模。原圖有向無向不可混淆，此構造與目標皆為無向圖。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

為何啟用會傳遍整條鏈

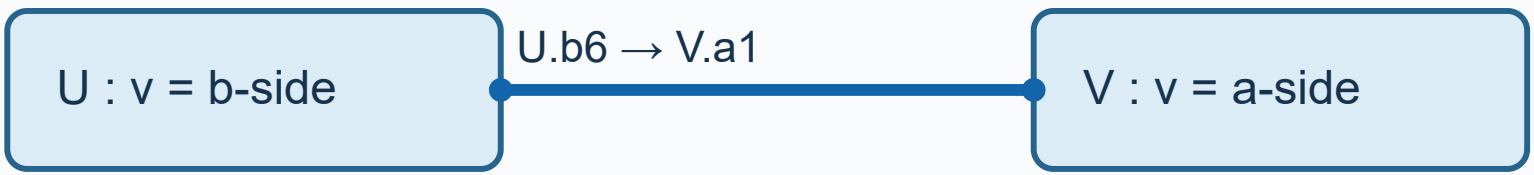
若 u 側兩個端口都使用外邊，稱此側為**啟用**。排序後的 u 鏈中，每個內部端口恰有一條可用外邊，即接相鄰 u 側的串鏈邊。若第 i 側啟用，它必使用該邊；相鄰側的端口因此使用外邊，由同一側兩個端口同時使用外邊的性質，該側另一端也必須使用外邊。向兩個方向歸納，u 的所有 incident gadgets 及鏈首尾都啟用。反之，一側未啟用，即不可能有任何一側啟用。

鏈的兩個端點在 gadget 外只接 selector；啟用的鏈因此在兩端各用一條 selector 邊。局部路徑連接同一 u 側兩端，故沿 cycle 行走時，抵達 selector 之前不會轉換原頂點身分。未啟用側的點仍被訪問：每個邊 gadget 至少有一側啟用；僅另一側啟用時，另一側的十二點完整路徑覆蓋所有點，包括未啟用側。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何啟用會傳遍整條鏈 （續）

例：u–v–w、k=1、cover={v}



此覆蓋使兩個 v 側啟用，以 U.b6 → V.a1 連接

未啟用 u、w 側的點仍在十二點完整路徑中

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

VC 到 HC 的完整 iff

若 C={u1,...,uk} 是 cover，將每個被選 u 的 incident-gadget 鏈作一條 cover path。原邊若只有 u 被選，使用從 u 側進出的十二點完整 path；若兩端都選，使用兩條各六點 path。因每條原邊至少一端被選，每個 gadget 的十二點恰好各被訪問一次。依序用 s1、u1 鏈、s2、u2 鏈、...、sk、uk 鏈、s1 串成一個 cycle，亦各訪 selector 一次，故為 HC。

反向由啟用沿 incident 串鏈傳遞的性質，任何啟用 u 側都強迫整條 u 鏈啟用；此鏈接兩個 selector，途中維持同一原頂點身分。取所有啟用的 u。每個 selector 在 cycle 中度數為二，每條啟用鏈消耗兩個 selector 端點，因此恰有 k 條鏈；k=1 時鏈首尾以兩條不同邊接回同一 selector。每條 u 鏈只有一對端點，不能重複計數。每個邊 gadget 的零啟用模式形成兩個封閉小環，因此至少一個端點的鏈啟用。所得 k 個頂點覆蓋所有原邊。

因此 VC(G,k) iff HC(G')；構造、反解與規模皆多項式。固定 yes/no 例處理前處理的退化情況，不需用不存在的空鏈湊 selectors。

VC gadget 的小例

原圖為 path u-v-w、k=1，可選 cover {v}。建兩個十二點 gadgets W_{uv}、W_{vw}；把 v 側第一個 gadget 的端口 6 接第二個 gadget 的端口 1，再用唯一 selector 接整條 v 鏈的首尾。

兩條原邊只有端點 v 被選：U 的 v 側為 b 側（x=1,y=0），V 的 v 側為 a 側（x=0,y=1）。串鏈邊 U.b6–V.a1 使兩側同時啟用；未啟用的 u、w 側頂點則包含在這兩條十二點完整路徑中。selector 與兩個 gadgets 共二十五點形成 HC。反之若選 u 的鏈，W_{vw} 沒有任何被選端點，無法覆蓋，因此不會假造 cover {u}。

完整二十五點cycle可由以下兩條十二點路徑接成。記U=W_{uv}、V=W_{vw}；U的b側代表v，V的a側代表v：

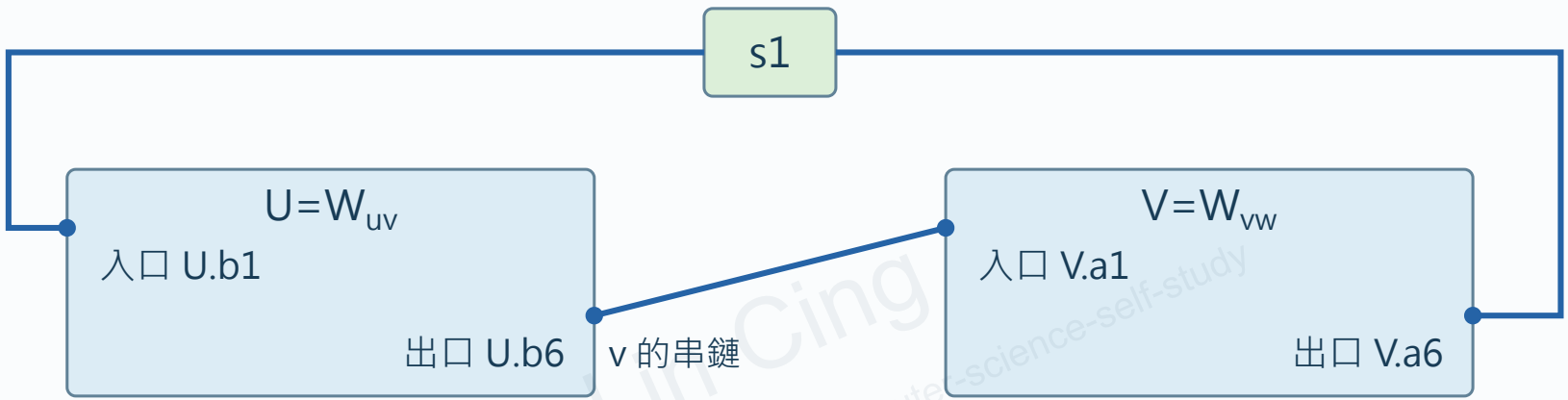
- U：b1,b2,b3,a1,a2,a3,a4,a5,a6,b4,b5,b6。
- V：a1,a2,a3,b1,b2,b3,b4,b5,b6,a4,a5,a6。

由s1→U.b1開始，走完U後經串接邊U.b6→V.a1，再走完V，最後V.a6→s1閉合。U與V的同名點是不同頂點，故24個gadget點加s1恰25點，每點只出現一次。每個相鄰pair都能在十四內邊、串鏈或selector邊中逐項查得。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

VC gadget 的小例 (續)

原圖 u–v–w、cover={v}：唯一 selector 串成25點 cycle



各框內是已列出的完整12點路徑，U與V的同名點互不相同。

此圖顯示選定的HC；完整外邊依selector與incident-chain構造定義。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

HAMILTONIAN cycle $\leq_p$ TSP decision

給 HC instance $G = (V, E)$，$n = |V|$。建立完全圖 K_n，邊權重：

$w(u, v) = 1$ 若 $\{u, v\} \in E$，否則 $w(u, v) = 2$。令門檻 $B = n$。

HC 到 tour。 G 的 Hamiltonian cycle 有 n 條原圖 edges，每條 weight 1，因此在 K_n 是 cost n 的 tour。

Tour 到 HC。 K_n 任一個 tour 恰有 n 條邊。若其 cost $\leq n$，且每條 weight 至少 1，則每條邊都必須 weight 1。這些邊都在 G，tour 因而也是 G 的 Hamiltonian cycle。

CLRS 原書將原邊設為 0、缺邊設為 1，門檻為 0；此教材對每條邊加 1。每個巡迴恰有 n 條邊，總成本都增加 n，因此門檻改為 n，yes/no 不變。

構造是 $O(n^2)$。所以 $\mathrm{HC}(G) \iff \mathrm{TSP}(K_n, w, n)$，且 TSP decision的 certificate 是一條 tour，檢查排列與總成本即可，故 TSP decision in NP。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TSP 實例追蹤

令G是三角形，edges={12,23,31}，n=3。完全圖的三條邊全部權重1，因此tour 1→2→3→1的成本3，門檻B=3。

門檻 B=3。此 tour cost 恰好等於 B，對應 G 的 Hamiltonian cycle。

若原圖刪去31，構造出的完全圖仍有31，但權重改為2。任何三點tour都含三條邊，成本1+1+2=4>B，所以不存在合格tour，與原圖沒有HC一致。對n<3的簡單無向圖，本章HC無解，直接映射至固定no TSP instance；上述每tour恰n邊的主要論證採n≥3。

optimization 版 TSP 要求真正最小總成本；decision 版只問是否存在 cost $\leq B$。若 optimization algorithm 在 polynomial time 回傳最佳值，決策版可直接比較 B；決策版 NP-hard 因而支持 optimization 版也難以精確 polynomial-time 求解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

SUBSET-SUM、bit encoding 與 DP

decision 版 SUBSET-SUM 問：給正整數集合 S 與 t，是否存在子集合元素和恰為 t。

Certificate 是一組 selected indices；verifier 檢查每個索引在範圍內、不重複，且所選整數和恰為 t。長度與檢查時間 polynomial，所以 SUBSET-SUM in NP。

DP 狀態 $dp[i, s]$ 表示前 i 個 elements 可否達和 s。轉移為

$dp[i, s] = dp[i-1, s] \lor (a_i \le s \land dp[i-1, s-a_i])$。

計算量 $O(nt)$。這是 pseudopolynomial，不是以 t 的 binary bit length 衡量的 polynomial。NP-hardness 的完整無進位數位表與兩向證明接在下頁；其巨大數值仍有多項式 bit 長，與此 DP 的 pseudopolynomial 界限一致。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3-CNF-SAT 到 SUBSET-SUM 的數位表

先把公式規格化為 n 個變數、k 個 clauses，每 clause 三個相異變數、無互補 tautology，每個變數至少出現一次；前面的去重/padding 處理可達成。恆真與恆假公式另輸出固定 yes/no 整數實例。

建立十進位 n+k 欄，前 n 欄對應變數，後 k 欄對應 clauses。這是緊湊建立整數的方式，最終仍以 binary 編碼交付。

- 每變數 x_i 建兩數 T_i、F_i。兩者的第 i 個變數欄為 1，其他變數欄為 0。T_i 在包含正 literal x_i 的 clause 欄為 1，F_i 在包含 $\neg x_i$ 的欄為 1，其餘為 0。
- 每 clause j 加 slack 數 S_j、S'_j，只在該 clause 欄分別放 1、2，其他欄為 0。
- target 的每個變數欄為 1，每個 clause 欄為 4。

總共 2n+2k 個正整數，每數 n+k 位十進位，故每數 binary 長度 $O(n+k)$，輸出總 bit 長 $O((n+k)^2)$，建構為多項式。若輸入要求集合而非 multiset，上述非退化假設保證兩個同變數數字不相同；不同變數的高位欄不同，slack 欄也不混同。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

為何無進位，以及兩向解碼

任意子集合在每個變數欄至多加到 2；每個 clause 含三個 distinct literals，對應 T_i/F_i 數最多提供三個 1，slack 最多加 1+2=3，因此 clause 欄至多 6<10。任何子集合都不會進位，不只是正確 witness 才不進位。

若原式可滿足，每變數依真假選 T_i 或 F_i，使變數欄和全為 1。每 clause 的真 literals 數 r 為 1、2 或 3，可用 slack 子集合湊 4-r：r=1 選 1+2，r=2 選 2，r=3 選 1。所有欄符合 target，得到 subset-sum yes。

反向若數字子集合和為 target，無進位迫使每變數欄恰為 1，所以 T_i、F_i 恰選一個，定義一致 assignment。clause target 為 4，但兩 slack 合計至多 3，所以至少一個被選 T_i/F_i 在該欄有 1，對應原 clause 中一個真 literal。每 clause 都滿足。兩向與多項式規模一起完成 NP-hardness，再加索引 certificate 的 NP membership，SUBSET-SUM 為 NP-complete。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

可手算的無進位例子

取 F=(x1∨x2∨x3) AND (¬x1∨x2∨¬x3)，欄序 x1,x2,x3,C1,C2：

數	十進位欄表示
T1,F1	10010，10001
T2,F2	01011，01000
T3,F3	00110，00101
S1,S1'	00010，00020
S2,S2'	00001，00002
target	11144

assignment x1=true,x2=true,x3=false 選 T1+T2+F3=11122。每個 clause 真 literal 數都是二，再選 S1'=20、S2'=2，得 11144。沒有進位；高位每欄的 1 也直接證明每個變數只選一個真值數。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

可手算的無進位例子 （續）

數值可能與 10^{n+k} 同階，但 binary 表示僅 $O(n+k)$ bits。這正是 reduction 可多項式輸出，而 $O(nt)$ 精確 DP 仍非輸入 bit 長度多項式的原因。

T1+T2+F3=11122；再加 slack 00022，得到 target 11144

	x1	x2	x3	C1	C2
T1	1	0	0	1	0
T2	0	1	0	1	1
F3	0	0	1	0	1
slack	0	0	0	2	2
target	1	1	1	4	4

變數欄各恰為1

兩個 clause 各有 2個真 literal

各以 slack 2 補到4

Lin Cing https://github.com/xinhi6716/computer-science-self-study

所有子集合都不能進位，才可反向解碼

若只對「依滿足指派選數」證明無進位，尚不足以證明反向：某個錯誤子集合也許靠進位偽造target。本構造對任意子集合都有變數欄≤2、clause欄≤6，故十進位下絕不進位。進位排除是所有候選的全稱命題。

每個數的位元長度O(n+k)，把十進位欄轉成binary可用逐欄乘10加digit的$O((n+k)^2)$簡單bit演算法；2n+2k個數也仍多項式。驗證子集合和時，至多加O(n+k)個O(n+k)位元整數，總和多出O(log(n+k))位元，亦多項式。

例如只取x1=false,x2=false,x3=false，兩clause的真literal數為0、2。第一欄即使兩個slack都取，也只有3，不能達4；不會向其他欄借出或借入一個位值來掩飾不滿足。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

近似保證比較的是所有輸入

NP-hard 最佳化仍可有多項式時間的近似解。本章每個演算法都先輸出可行解，再證明它與未知 OPT 的關係。單一測資接近最佳值不能證明近似比。

核心工具為 matching 下界、MST 下界、元素收費與動態規劃狀態修剪。前置知識為貪婪法、MST、歸納法及二進位輸入長度。

來源對照為 CLRS 3e §§35.1–35.3、35.5；MIT 2004 lecture 24；手寫筆記 PDF 54–56。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最小化與最大化的界不能倒置

設 A 為可行解值，OPT 為最佳值，$\rho \geq 1$。正值目標下的統一比率為 $\max(A/OPT, OPT/A) \leq \rho$。

問題	可行性給的界	近似保證
最小化	$OPT \leq A$	$A \leq \rho OPT$
最大化	$A \leq OPT$	$A \geq OPT/\rho$

例如最小化 OPT=10、A=15 是 1.5 倍；最大化 OPT=15、A=10 也是 1.5 倍。最大化寫 $A \leq \rho OPT$ 幾乎沒有內容。OPT=0 時避免除法，直接用乘法式；subset sum 無可用正整數時回傳 0。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

頂點覆蓋每輪選邊的兩端

無向圖 G=(V,E) 的 vertex cover C 必須碰到每一條邊，目標最小化 |C|。令 C=∅、R=E。當 R 非空，任選 uv∈R，把 u,v 加入 C，刪掉 R 中所有與 u 或 v 相接的邊。

終止時每條原始邊都因為已有端點在 C 才被刪除，因此 C 可行。可用一次掃描產生 maximal matching，再取全部端點，時間 O(|V|+|E|)。此演算法不直接處理帶權頂點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

頂點覆蓋每輪選邊的兩端 （續）

頂點覆蓋每輪兩端點

a — b — c — d — e — f

1: ab → C={a,b}; R={cd,de,ef}

2: cd → C={a,b,c,d}; R={ef}

3: ef → |C|=6; |M|=3; OPT=3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐輪記錄把演算法與證明接起來

取 E={ab,bc,cd,de,ef}，依序挑 ab、cd、ef。

輪	挑邊	新增 C	刪除的剩餘邊	輪後剩餘
1	ab	a,b	ab,bc	cd,de,ef
2	cd	c,d	cd,de	ef
3	ef	e,f	ef	∅

M={ab,cd,ef} 是 matching；C={a,b,c,d,e,f}。覆蓋 {b,d,f} 的大小為 3。三條互不相接的邊又要求任何覆蓋至少 3 點，故 OPT=3，A/OPT=6/3=2。這是確實達到 2 的例子。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

2 倍證明依靠 matching 下界

令 M 為各輪挑出的邊。選 uv 後立即刪掉所有相接邊，所以往後不可能再選含 u 或 v 的邊，M 的邊兩兩不共端點。

任一覆蓋 C* 都要覆蓋 M 每條邊。因端點互不重疊，這些邊不能共用同一個覆蓋頂點，故 $|C^*| \geq |M|$。演算法每條邊恰選兩個新端點，所以 $|C| = 2|M| \leq 2|C^*|$。

M 為 maximal，因終止後沒有邊可再加入；不必是 maximum。路徑 a-b-c-d 中 {bc} maximal 但大小 1，maximum 為 {ab,cd}、大小 2。只取每條 matching 邊任意一端，不能保證覆蓋其他邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Metric TSP 先明定圖與成本

輸入為至少三個頂點的完整無向圖，成本非負、對稱，且任意 u,v,w 滿足 $c(u,w) \le c(u,v)+c(v,w)$。目標為每點恰訪一次再回起點的 Hamiltonian cycle。

計算 MST T，任選根，沿樹做完整 DFS 走訪 W，每條樹邊去回各一次。保留每個頂點第一次出現的次序，最後回根，得到 tour H。

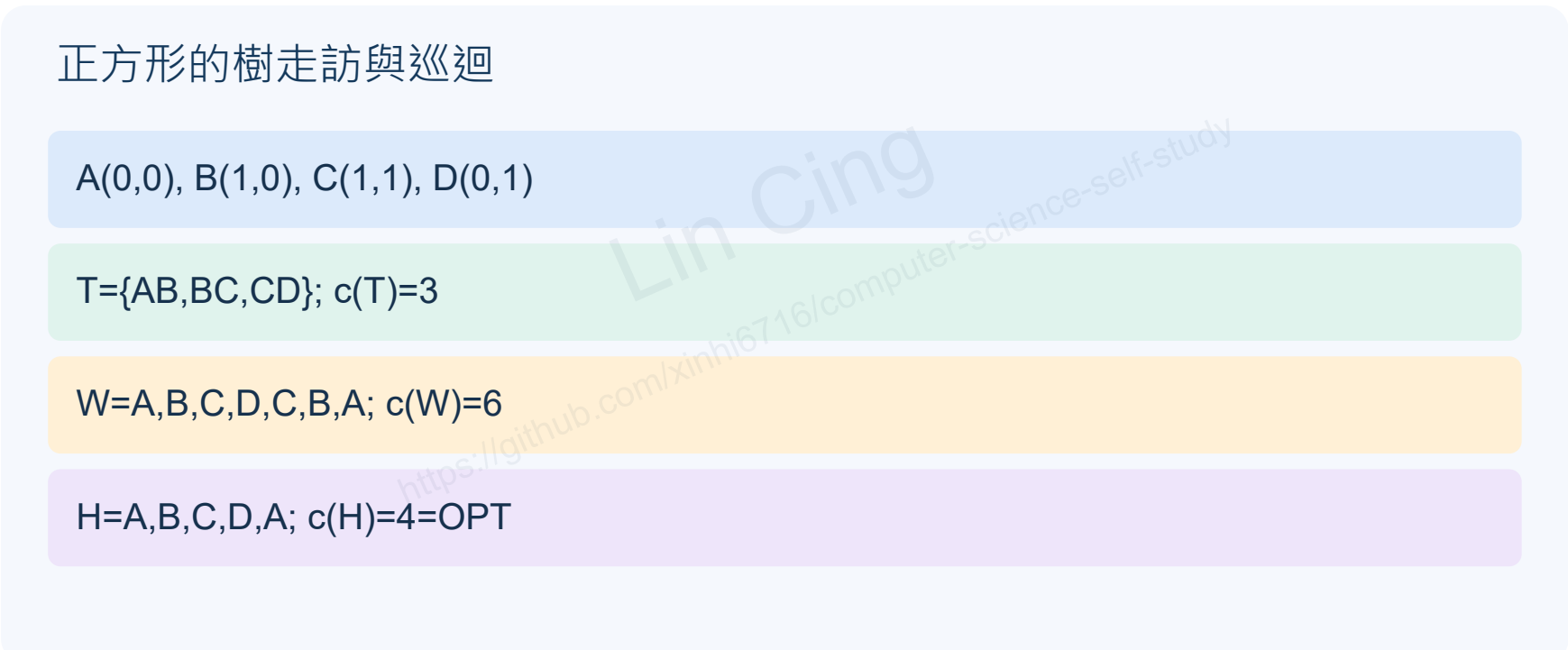
完整走訪 W 常重訪根或分支點，因此 W 只是 closed walk，尚未是可行的 TSP tour。完整圖確保所有捷徑邊存在。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

樹走訪到 tour 的數值範例

四點正方形 A=(0,0)、B=(1,0)、C=(1,1)、D=(0,1)，使用 Euclidean 距離。MST 可選 AB、BC、CD，成本 3。

完整走訪 A,B,C,D,C,B,A 成本 6；保留第一次出現並回根，得 A,B,C,D,A，成本 4。任何閉合巡迴至少有四條邊，每條距離至少 1，故 OPT≥4；此 tour 又達 4，OPT=4。



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Double-tree 的三個不等式

從最佳 tour H^* 刪去任一邊，得到 spanning tree。非負成本使刪邊不增成本，MST 最小性給 $c(T) \le c(H^*)$。

W 每條樹邊走兩次，所以 $c(W) = 2c(T)$。對 W 中兩次保留頂點間的路段 $u = v_0, v_1, \dots, v_k = w$，反覆用三角不等式可得 $c(u,w) \le \sum_j c(v_{j-1}, v_j)$。所有這些路段把 W 分割，故捷徑後 $c(H) \le c(W)$。

合併即 $c(H) \le c(W) = 2c(T) \le 2\mathrm{OPT}$。密集完整圖可用 $O(n^2)$ 的 Prim，走訪與取首次出現需 $O(n)$，總計 $O(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

少了三角不等式，捷徑可能更貴

令四點完整圖 AB=BC=CD=DA=1，AC=BD=M>4。MST 可選以 A 為根的 AB、AD、BC，先訪 D 再訪 B 的 preorder 為 A,D,B,C，tour 成本 2+2M；最佳 tour A,B,C,D,A 成本 4。

完整走訪 A,D,A,B,C,B,A 成本 6。shortcut D→A→B 改為 D→B，成本由 2 增至 M；最後 C→B→A 改 C→A 也增加。比率 (2+2M)/4 可任意大。M=10 時為 22/4=5.5。

這證明 double-tree 保證會失效。更強的「任意常數近似皆不存在，除非 P=NP」另需 gap reduction，不能只靠此反例推出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

集合覆蓋貪婪選最多新元素

給 universe U 與集合族 F，假設 $\cup F = U$；選最少集合覆蓋 U。每輪選 $|S \cap R|$ 最大的 S，R 為尚未覆蓋元素，直到 $R = \varnothing$。平手按 A,B,C 的名稱順序。

例 $U = \{1, \cdots, 6\}$，$A = \{1,2,3,4\}$、$B = \{1,2,5\}$、$C = \{3,4,6\}$。先選 A 新增 4 點，接著 B 新增 5，最後 C 新增 6。貪婪共 3 組；$B \cup C = U$ 且沒有單一全集，故 $OPT = 2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

集合覆蓋貪婪選最多新元素 （續）

首次覆蓋的收費帳本

A={1,2,3,4}; B={1,2,5}; C={3,4,6}

A → 1,2,3,4: each 1/4

B → 5: 1; C → 6: 1

Total=3; OPT=2; H4=25/12

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每輪成本平均分給首次覆蓋元素

一輪新增 k 個元素，就給這 k 個元素各收 $c_x=1/k$。每元素只收一次，因此貪婪集合數 $|G|=\Sigma_x c_x$。

上例 $c_1=c_2=c_3=c_4=1/4$，$c_5=c_6=1$。總收費 $4\times 1/4+1+1=3$。最佳組 B 的收費 $1/4+1/4+1=3/2$，C 也是 3/2。

對任意最佳覆蓋 F*，每元素至少被一組包含，且收費非負，故 $\sum_x c_x \le \sum_{S\in F^*}\sum_{x\in S} c_x$。重複計費只使右側更大。剩下要證明每組的總收費有 harmonic 上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Harmonic charging 的完整局部論證

固定任一 S∈F，令 s=|S|。依被首次覆蓋的先後排列其元素 $x_1,\cdots,x_s$，同輪任意排序。

x_j 被覆蓋那輪開始時，至少 s−j+1 個 S 元素未被覆蓋。即使 x_j 前面有同輪元素，這個數仍是下界。貪婪選中的集合新增 k≥|S∩R|≥s−j+1 點，否則改選 S 更好。

所以 $c_{x_j} = 1/k \le 1/(s-j+1)$。加總得 $\sum_{x\in S} c_x \le \sum_{j=1}^{s} 1/(s-j+1) = H_s$。這個界適用所有候選集合，而不只貪婪選到的集合。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

由局部收費得到全域近似比

若 U=∅，貪婪與最佳解皆為空集合族，OPT=0；成本不等式成立，不計算 0/0，也不取空集合的最大值或 ln 0。以下假設 U≠∅，令 $H_0=0$、$d=\max_{S\in F}|S|\ge 1$。上一頁與最佳覆蓋的重複計費不等式給出

$|G|=\sum_x c_x \le \sum_{S\in F^*}\sum_{x\in S} c_x \le \sum_{S\in F^*} H_{|S|} \le H_d|F^*|$。

因 $H_d=1+\sum_{j=2}^{d}1/j \le 1+\int_1^d dx/x = 1+\ln d \le 1+\ln|U|$，這是 H_d 近似。例中 d=4、$H_4=25/12$，保證 $3\le(25/12)\times 2=25/6$；實際比率為 3/2。

簡單重掃實作每輪 $O(mN)$、至多 N 輪，為 $O(mN^2)$，m=|F|、N=|U|。這已足夠證明多項式時間；不把此界誤寫為 log N 無常數項。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Subset sum 的精確列表為何爆炸

輸入正整數 $x_1,\dots,x_n$ 與整數目標 $t \ge 1$，找總和不超過 t 的最大值。允許空集合。精確狀態 $P_0=\{0\}$，$P_i=P_{i-1}\cup(P_{i-1}+x_i)$，只保留 $\le t$ 的值。

每個值都能追溯到實際子集合，但列表可能有 2^n 個值。$O(nt)$ 的布林 DP 只是 pseudo-polynomial，因二進位 t 的長度是 $\Theta(\log t)$。

固定 $0<\varepsilon<1$，令 $\delta=\varepsilon/(2n)$。近似列表 L_i 只保留相鄰數值相差足夠大的代表，目標為 $\mathrm{OPT}/(1+\varepsilon) \le \max L_n \le \mathrm{OPT}$。若 $n=0$ 或沒有任何 $x_i \le t$，直接回傳 0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trim 保留較小代表以維持可行性

對已排序去重列表，先保留最小值。掃描 y 時，僅在 $y>(1+\delta)\text{last}$ 才保留 y 並更新 last；否則由 last 代表 y。每個原值 y 都有保留值 z 滿足 $y/(1+\delta)\le z\le y$。

每輪先合併 L_{i-1} 與 $L_{i-1}+x_i$，再 TRIM，最後刪掉 $>t$ 的值。始終保留 0。正整數的 0 需獨立處理，不能拿 log 0。

用較小代表確保未來加相同的非負 x_i 後仍不會超過對應真實和。保留值本身仍是實際子集合和；若需輸出子集合，可附帶前驅狀態與選物索引。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trim 保留較小代表以維持可行性 （續）

修剪列表 δ=0.05, t=308

L1={0,104}

L2={0,102,206}

L3={0,102,201,303}

L4={0,101,201,302}; OPT=307

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

修剪小例逐輪驗算

取 x=(104,102,201,101)、t=308、ε=0.4，n=4、δ=0.05。用精確比較避免浮點邊界。

i	合併後	修剪並刪掉 >308 後
1	0,104	0,104
2	0,102,104,206	0,102,206
3	0,102,201,206,303,407	0,102,201,303
4	0,101,102,201,203,302,303,404	0,101,201,302

第二輪 104≤102×1.05=107.1 被刪；第三輪 206≤201×1.05=211.05 被刪；第四輪 303≤302×1.05=317.1 被刪。結果 302=201+101，最佳為 307=104+102+101。比率 307/302≈1.01656≤1.4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

誤差沿 n 輪相乘的歸納證明

主張每個 $y \in P_i$、$y \le t$，都有 $z \in L_i$ 使 $y/(1+\delta)^i \le z \le y$。$i=0$ 僅有 0，成立。

若 y 未使用 x_i，由歸納假設取上一輪代表 q。若 $y=p+x_i$，取 p 的代表 q，有 $q+x_i \ge p/(1+\delta)^{i-1}+x_i \ge (p+x_i)/(1+\delta)^{i-1}$，因 $x_i \ge 0$。兩種情況都在合併列表找到 $w \in [y/(1+\delta)^{i-1}, y]$。

本輪修剪留下 $z \in [w/(1+\delta), w]$，得到主張。又 $z \le y \le t$，所以不會被容量刪除。套用 $y=\mathrm{OPT}$，最大保留值 $A \ge \mathrm{OPT}/(1+\delta)^n$。

$\ln(1+\delta) \le \delta$，故 $(1+\delta)^n \le e^{\{\varepsilon/2\}} \le 1+\varepsilon$。最後一步由 $f(\varepsilon)=\ln(1+\varepsilon)-\varepsilon/2$，$f(0)=0$、$f'(\varepsilon)=1/(1+\varepsilon)-1/2 \ge 0$，適用 $0 \le \varepsilon \le 1$。故乘法式 $A \ge \mathrm{OPT}/(1+\varepsilon)$ 對所有情形成立。若 $\mathrm{OPT}=0$，由可行性得 $A=0$，不取比率；若 $\mathrm{OPT}>0$，前述下界給出 $A>0$，才可除以 A 得 $\mathrm{OPT}/A \le 1+\varepsilon$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

FPTAS 必須對位元長度與 1/ε 多項式

修剪後的正值相鄰比 >1+δ，且全在 [1,t]。所以 $|L_i| \le 2+\ln t/\ln(1+\delta)$。因 $\ln(1+\delta) \ge \delta/(1+\delta)$，$\delta=\varepsilon/(2n)<1/2$，故 $|L_i| \le 2+3n \ln t/\varepsilon$。

設 $K=O(1+n \log t/\varepsilon)$。用兩個有序列表線性合併，每輪 $O(K)$ 次算術操作，n 輪為 $O(nK)$。若改用一般排序，額外 $O(\log K)$ 因子仍為多項式。

二進位輸入長度 B 包含各 x_i、t 與有理 ε 的編碼。列表整數需要 $O(\log t)$ 位元，合併前至多 $t+\max x_i$；有理比較可交叉相乘，因此位元運算也對 B 與 1/ε 多項式。ε 不視為免費的無限精度實數。

PTAS 只要求固定 ε 時對 B 多項式，例如 B^{1/ε}；FPTAS 另要求對 1/ε 多項式。本章修剪法滿足後者。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

加權集合覆蓋：按新增元素的單價選擇

給定集合族覆蓋有限 universe U，每個集合成本為正有理數。令 R 是尚未覆蓋元素；每輪選 $w(S)/|S \cap R|$ 最小且分母不為零的集合。空 U 回傳空解；若集合聯集不含 U，回報無解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐輪追蹤與反例

$U=\{1,2,3\}$；$A=U$、$B=\{1,2\}$、$C=\{3\}$，成本分別為 9、2、1。比值為 3、1、1。若先選 B，它的兩元素各付 1；再選 C，元素 3 付 1，總成本 3。B, C 合起來成本 3，含 A 的方案至少成本 9，故 OPT=3。單純選新增元素最多會先選 A，成本 9；若將其成本換成任意大的 M，比值即無界。

加權覆蓋：集合與逐輪收費

A: {1 2 3} 成本 9

B: {1 2} 成本 2

C: {3} 成本 1

第 1 輪：選 B | 2 / 2 = 1

元素 1 收費 1；元素 2 收費 1

第 2 輪：選 C | 1 / 1 = 1

元素 3 收費 1

演算法成本 ALG = 3；最優 OPT = 3

Lin Cing https://github.com/xinhi6716/computer-science-self-study

每個元素的收費構成局部上界

每輪選集合 S、新覆蓋 k 個元素，將 $w(S)/k$ 分給這些元素。每個元素恰首次被覆蓋時付費，所以收費總額等於 ALG。固定任意集合 A，將其中元素按首次覆蓋排序為 $e_1, \ldots, e_t$。覆蓋 e_i 的那一輪開始時，A 至少還有 $t - i + 1$ 個未覆蓋元素。即使幾個元素同輪覆蓋，順序任排也成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把局部上界加到最優解

因貪婪法在該輪可選 A，$charge(e_i) \le w(A)/(t-i+1)$。加總得 $\sum_{e\in A} charge(e) \le w(A) H_{|A|}$。設 $d = \max_S |S|$，最優覆蓋至少含每個元素一次。收費非負，所以重複計費只放大右側：

$ALG \le \sum_{A\in OPT} \sum_{e\in A} charge(e) \le H_d \, OPT$，其中 $H_d = \sum_{i=1}^{d} 1/i \le 1 + \ln d$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MAX-3-CNF：期望保證

每句含三個相異變數；各變數獨立公平取值。每句八種取值中只有一種使它為假，故滿足機率為 $7/8$。設 Y 為滿足句數、m 為總句數，期望線性給 $E[Y] = 7m/8 \geq 7OPT/8$；句子之間不必獨立。單句仍有 $1/8$ 機率不滿足，故此界不是每次輸出的保證。重複文字 $(a \vee a \vee a)$ 的滿足率只有 $1/2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

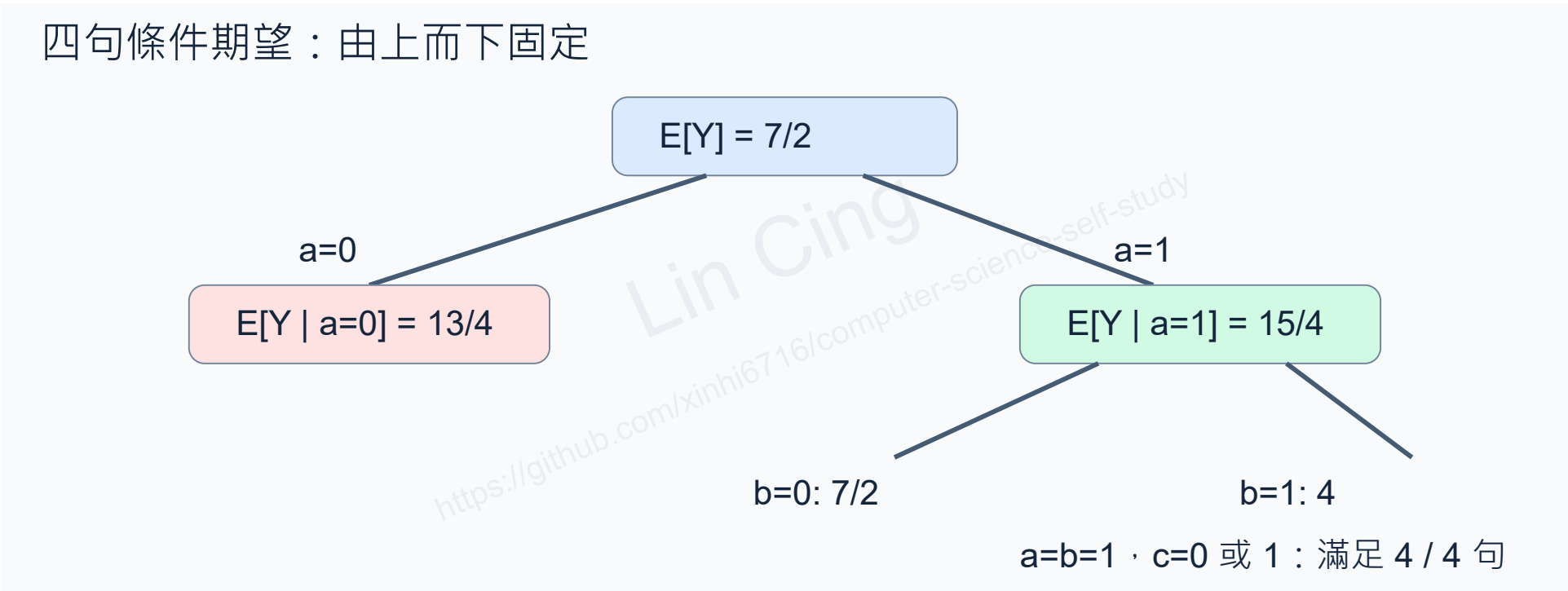
逐一固定變數，不降低條件期望

在部分指派下，已滿足句的貢獻為 1，全指派且為假的貢獻為 0；其餘若有 r 個未定變數，貢獻為 $1-2^{-r}$。下一變數固定為 0 或 1 的兩個條件期望平均等於目前期望。選較大者，再重複；固定完畢後，條件期望就是實際滿足句數，至少 $7m/8 \geq 7OPT/8$。每步直接重算全部句子需 $O(nm)$ 時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

條件期望的四句數例

四句依序為 $(a \vee b \vee c)$、$(\neg a \vee b \vee c)$、$(a \vee \neg b \vee c)$、$(a \vee b \vee \neg c)$。初始期望 $7/2$。固定 $a = 0$ 或 1 得 $13/4$ 或 $15/4$，選 1；接著固定 $b = 0$ 或 1 得 $7/2$ 或 4，選 1；c 任取均滿足四句。



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

加權頂點覆蓋：整數模型與 LP

無向圖頂點具有非負有理權重。以 $x_v \in \{0,1\}$ 表示選取，最小化 $\sum_v w_v x_v$，每條邊 uv 要求 $x_u + x_v \geq 1$。改為 $0 \leq x_v \leq 1$ 得 LP；每個整數解仍可行，故 $LP^* \leq OPT$。以多項式時間有理 LP 方法求精確最優解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

門檻捨入為何可行且不超過兩倍

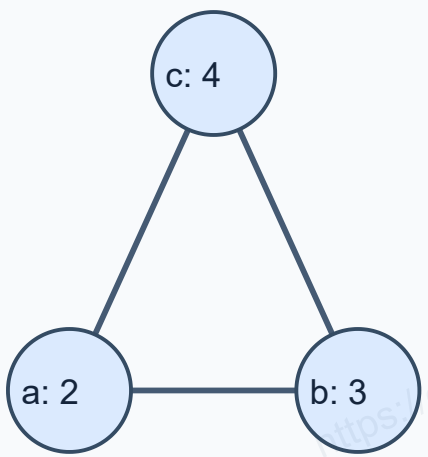
選所有 $x_v \geq 1/2$ 的頂點。若某邊兩端皆未選，兩端值都小於 $1/2$，總和小於 1，與 LP 限制矛盾。逐頂點 $1_{v \in C} \leq 2x_v$；乘非負權重並加總，得 $w(C) \leq 2LP^* \leq 2OPT$。單邊兩端權重為 1 和 M 時，直接取 matching 兩端成本 $1 + M$、OPT=1，不能套用未加權法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三角形：下界證書與捨入

三角形 a, b, c 權重為 2、3、4。半值解成本 $9/2$。將 ab, ac, bc 三邊限制分別乘 $1/2, 3/2, 5/2$ 再相加，得到任何 LP 可行解成本至少 $9/2$，所以半值解為最優。門檻捨入選三點成本 9；整數最優取 a, b，成本 5，實際比值 $9/5$。

三角形 LP 與證書



$x_a = x_b = x_c = 1/2$

ab * 1/2 + ac * 3/2 + bc * 5/2

$2x_a + 3x_b + 4x_c \geq 9/2$

LP 最優值 = 9/2；捨入門檻 1/2

{a,b,c} 成本 9；{a,b} 最優成本 5

Lin Cing https://github.com/xinhi6716/computer-science-self-study

隨機捨入：先處理漏覆事件

令 $N=|U|\ge 2$、成本非負，LP 最優解滿足每元素 $\sum_{S\ni e} x_S \ge 1$。取 $k=3\lceil \log_2 N\rceil$，每集合獨立以 $p_S=\min(1,kx_S)$ 選取。若某包含 e 的集合有 $p_S=1$，它必被覆蓋；否則 $\Pr(e\text{漏覆})=\prod_{S\ni e}(1-p_S)\le e^{-k}\le N^{-3}$。聯集界給任一元素漏覆的機率至多 $N^{-2}\le 1/4$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

成本與可行性要同時檢查

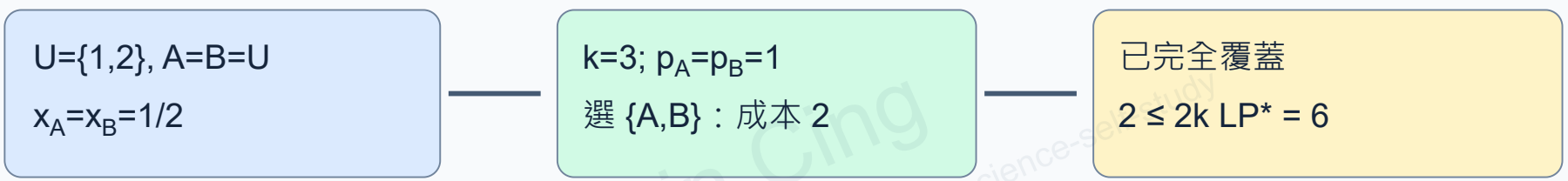
期望成本 $\sum_S w_S p_S \le kLP^*$。若 $LP^* > 0$，Markov 不等式給成本超過 $2kLP^*$ 的機率至多 $1/2$。加上漏覆機率，單次任一失敗機率至多 $3/4$，所以成功機率至少 $1/4$。檢查兩條件，失敗就獨立重試；期望試驗次數至多 4，且成功成本 $\le 2kOPT$。$LP^* = 0$ 時成本必為 0，僅需重試覆蓋；$N = 0$ 回空解，$N = 1$ 選最便宜的覆蓋集合。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一個隨機抽樣步驟

令 $U=\{1,2\}$，$A=B=U$ 且各成本 1。取最優 LP 解 $x_A=x_B=1/2$，$LP^*=1$，$k=3$，故 $p_A=p_B=1$。兩者都選，成本 2 且完整覆蓋，滿足 $2 \le 2kLP^*=6$。此例示範截斷至 1；不是失敗機率界的緊例。

隨機捨入：截斷、抽樣及檢查



漏覆事件機率 ≤ 1/4

成本超過 2k LP* 的機率 ≤ 1/2

聯集界：成功機率 ≥ 1/4；失敗則重試

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非 metric TSP：建構有間隙的輸入

假設存在固定 $\rho \geq 1$ 的確定性多項式近似法，取固定整數 $R \geq \rho$。把 n 點 Hamiltonian cycle 輸入 G 轉為完全圖：原邊成本 1，非邊成本 $M = Rn + 1$。若 G 有 Hamiltonian cycle，$OPT = n$，近似輸出成本至多 Rn。若沒有，任何 tour 至少使用一條非邊，成本至少 $M + n - 1 > Rn$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

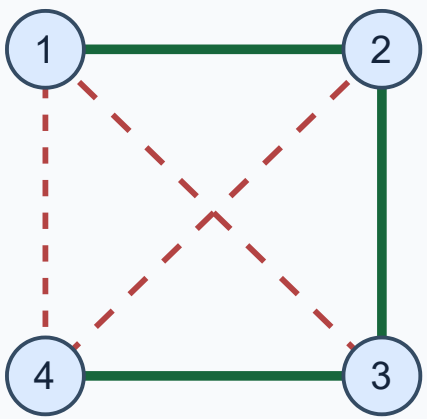
成本門檻會判定 Hamiltonian cycle

用近似法輸出是否 $\leq Rn$ 判定原圖有無 Hamiltonian cycle，所以該近似法存在將推出 $P = NP$。M 的二進位長度為 $O(\log n)$，完全圖輸出長度多項式。$n = 4, R = 2$ 時 $M = 9$；若原圖是四點路徑，缺邊成本 9，每個 tour 至少一條缺邊，成本至少 12，高於門檻 8。這些成本可能違反三角不等式；證明也不直接排除僅有期望保證的隨機演算法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

成本門檻會判定 Hamiltonian cycle （續）

路徑變完全圖：成本落差



n=4, R=2, M=Rn+1=9

綠色實線：原邊成本 1

紅色虛線：非邊成本 9

路徑 1-2-3-4-1：成本 12

無 Hamilton 圈：每條巡迴 ≥ 12 > 8

有 Hamilton 圈：近似輸出 ≤ 8

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三文字子句：期望與單次成功機率

設 $m \geq 1$，每個 clause 恰有三個相異變數（可有否定）。各變數獨立均勻賦值，每 clause 恰有一種使它為假的局部指派，故 $\mathbb{E}[X] = 7m/8$，其中 X 是滿足子句數。子句之間即使共享變數，期望的線性仍成立；這裡並未假設子句滿足事件獨立。

設 $q = \Pr(X \geq 3m/4)$。由 $0 \leq X \leq m$，在成功時用 m 上界、失敗時用 $3m/4$ 上界：

$$\frac{7m}{8} = \mathbb{E}[X] \leq qm + (1-q)\frac{3m}{4}, \qquad q \geq \frac{1}{2}.$$

因 $OPT \leq m$，成功時同時有 $X \geq 3OPT/4$。這是成功機率下界，不是每次都成功；若有一文字或重複變數子句，$7/8$ 的前提可能失效。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

可檢查的重試與小例子

獨立重做 r 次並保留最多滿足子句的指派，達不到 $3m/4$ 的機率至多 2^{-r}；每次都能直接計數檢查，無須知道 OPT。例：一個 clause $(a \vee b \vee c)$，8 種指派中 7 種滿足，因此 $X \geq 3/4$ 的機率是 $7/8$，高於通用下界 $1/2$。$m = 0$ 直接回傳任意指派；不存在比值分母問題。

這是「以高機率得到至少 $3OPT/4$」的保證。期望近似比、單次成功率與每次必然的保證不同；若需要確定性結果，可以採用逐變數最大化條件期望的另一路做法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Christofides：目標與輸入假設

輸入是 $n \geq 3$ 的完整無向圖；邊成本非負、對稱，且對所有頂點滿足三角不等式 $c(u, w) \leq c(u, v) + c(v, w)$。允許不同頂點距離為零。目標是最小成本 Hamilton 巡迴；令其成本為 OPT。以下演算法回傳可行巡迴，成本至多 $3OPT/2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從樹到偶度連通多重圖

1. 求最小生成樹 T，收集其中奇度頂點集合 O。
2. 在 O 的完整子圖求**最小權完美配對** M。
3. 將 T 和 M 合併成多重圖，相同邊若同時出現在兩者，保留兩份。
4. 沿所有邊走一次求 Euler 閉行走，之後略過已見頂點並回起點。

握手定理給 $|O|$ 為偶數。每個 O 頂點在配對中恰加一條邊，度數由奇轉偶；其他點不變。因 T 連通，合併圖連通且每點偶度，故存在 Euler 閉行走。一般圖最小權完美配對可多項式時間求得；任意配對不保證以下成本界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第一個下界：樹不比最佳巡迴貴

由最佳巡迴刪除任一邊得到生成樹。邊成本非負，刪邊不增加成本；最小生成樹不比該樹貴，所以 $c(T) \leq OPT$。若容許負邊，這一步不能照用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第二個下界：從最佳巡迴抽出奇度點

按照最佳巡迴中遇到 O 的順序，將其餘頂點逐一略過。完整圖提供每條捷徑，反覆使用三角不等式得 O 上的封閉循環成本至多 OPT。生成樹在 $n \geq 3$ 時至少有兩個奇度點。若 $|O| = 2$，這個循環視為兩點間往返，各方向各一份邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

交替拆成兩個完美配對

奇度點數為偶數。把上述循環的邊按順序交替分成兩組；每組都讓每個奇度點恰好出現一次，因此是完美配對。兩組成本之和不超過 OPT，所以較便宜的一組成本至多 $OPT/2$。最小權配對 M 不比它貴，故 $c(M) \leq OPT/2$。兩點時兩份往返邊各構成同一條邊的配對，論證仍成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

捷徑保持可行且不增成本

Euler 閉行走總成本恰為 $c(T) + c(M)$。按首次出現順序保留每個頂點，其餘走訪由直接邊取代；每次略過一段路都可由三角不等式證明不增成本。最後回起點，得到每個頂點恰好一次的巡迴。故

$c(\text{tour}) \leq c(T) + c(M) \leq OPT + OPT/2 = 3OPT/2$。

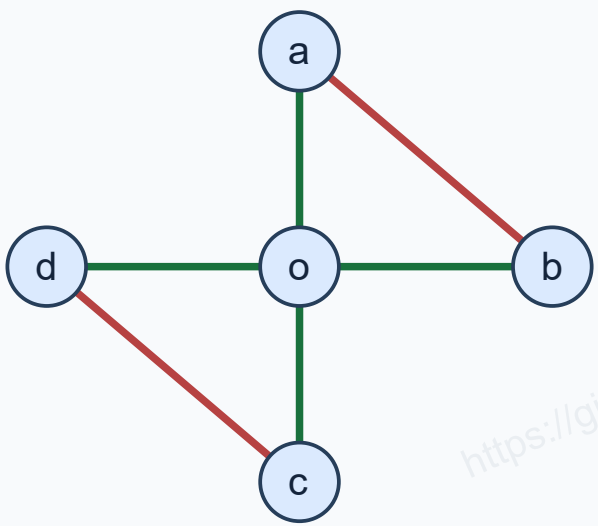
零成本 OPT 時，這個乘法不等式仍成立，不需除以 OPT。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四葉星形 metric：樹、配對與實際巡迴

五點 o, a, b, c, d：中心到各葉距離 1，兩葉距離 2。這是樹路徑距離，滿足三角不等式。星形四邊總成本 4，是 MST：每個生成樹有四邊且每邊成本至少 1。中心度 4，四葉度 1，所以 $O = \{a, b, c, d\}$。各葉對距離均為 2，選 $M = \{ab, cd\}$ 成本 4。

星形 MST、配對與 Euler 行走



綠線：MST，四邊各成本 1

紅線：配對 ab、cd，各成本 2

Euler：o → a → b → o → c → d → o

成本 1+2+1+1+2+1 = 8

略過第二次 o：o → a → b → c → d → o

巡迴成本 8 = OPT

Lin Cing https://github.com/xinhi6716/computer-science-self-study

數例的 Euler 行走與 OPT

合併圖可走 $o \to a \to b \to o \to c \to d \to o$，成本 $1+2+1+1+2+1=8$。略過第二次出現的 o 得 $o \to a \to b \to c \to d \to o$，成本 $1+2+2+2+1=8$。任何五點巡迴在中心有兩條成本 1 的邊，其餘三邊連兩葉、各成本 2，所以 OPT=8。此例輸出最優；$3/2$ 是對所有符合假設的輸入的上界，不是此例的實際比值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd edition, chapters 34 and 35. MIT 6.046J Fall 2004, Lecture 24, approximation algorithms. Original handwritten algorithms notes, complexity and approximation pages.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study